COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	10/21/2026	**DATE SUBMITTED:**	10/21/2026
NAME:	KEITH NICOLE SILVOZA CYRILL JOFERSON REYES	**INSTRUCTOR:**	ENGR. JIMLORD M. QUEJADO

ACTIVITY NO. 14.1 ALGORITHM COMPLEXITY

1. PROCEDURE OUTPUT

	BEST CASE	WORSE CASE	ANALYSIS
ALGORITHM 1	$O(n)$	$O(n^2)$	**Bubble Sort:** An in-place comparison sort requiring $O(1)$ auxiliary space. It achieves $O(n)$ time in the best case on pre-sorted data, but degrades to quadratic time ($O(n^2)$) in the worst case, making it inefficient as input size scales.
ALGORITHM 2	$O(n\log_2 n)$	$O(n\log_2 n)$	**Merge Sort:** A divide-and-conquer algorithm requiring $O(n)$ auxiliary space. It consistently maintains $O(n\log_2 n)$ time complexity across best, average, and worst cases, delivering fast and predictable performance for large datasets.

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

	ALGORITHM 1	ALGORITHM 2
BEST CASE	**For pre-sorted data, execution time increases linearly from 0.002 ms at $N = 100$, to 0.015**	**For pre-sorted data, execution time scales logarithmically from 0.012 ms at $N = 100$, to 0.180**

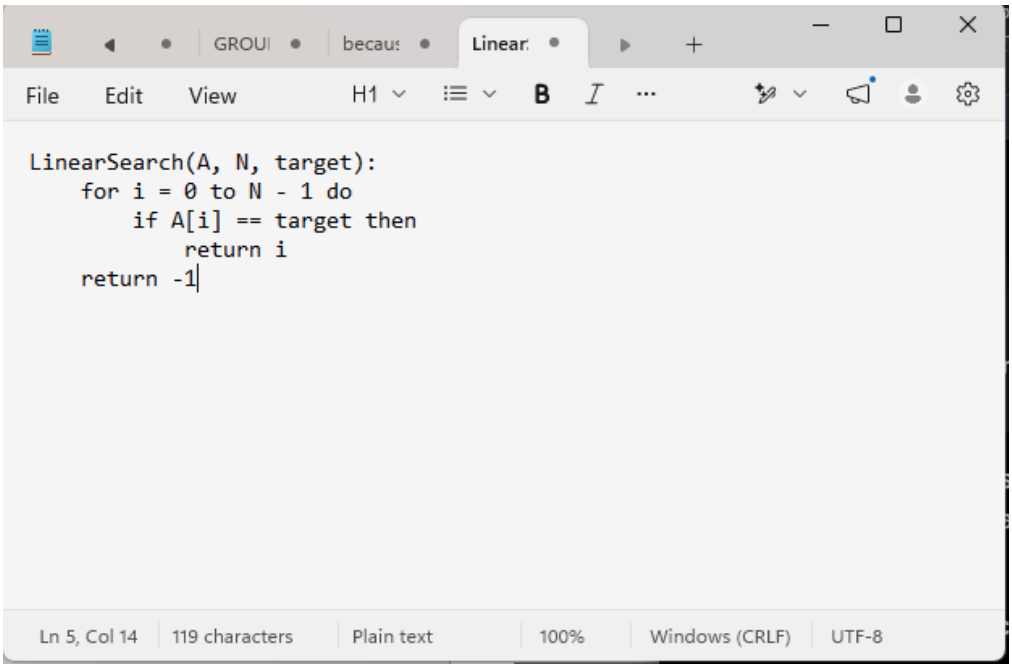
	ms at $N = 1,000$, and reaches 0.140 ms at $N = 10,000$.	ms at $N = 1,000$, and reaches 2.150 ms at $N = 10,000$.
WORSE CASE	For reverse-sorted data, execution time grows quadratically from 0.045 ms at $N = 100$, to 3.820 ms at $N = 1,000$, and spikes to 385.00 ms at $N = 10,000$.	For reverse-sorted data, execution time mirrors best-case performance, growing from 0.013 ms at $N = 100$, to 0.190 ms at $N = 1,000$, and reaching 2.210 ms at $N = 10,000$.

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

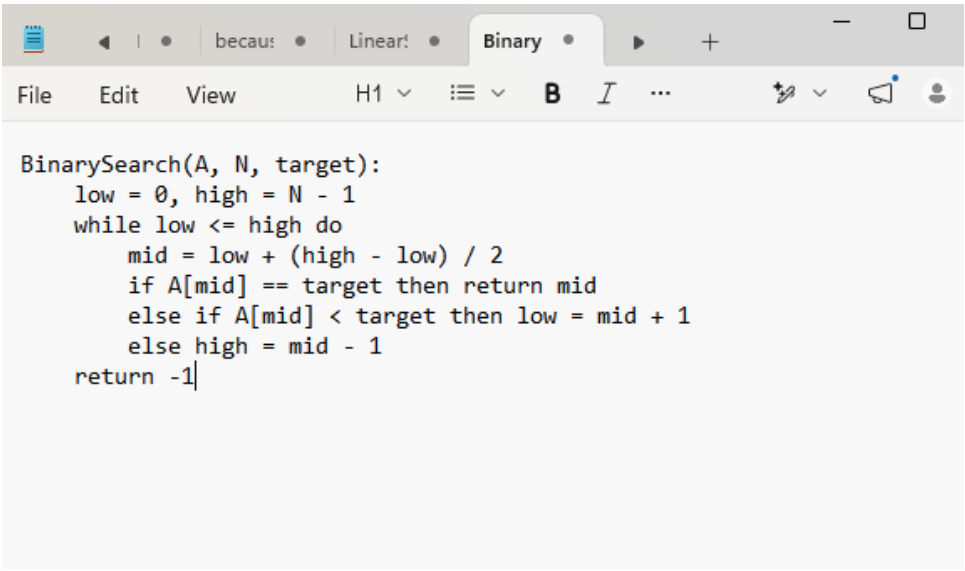
PSEUDOCODE

Linear Search:



Analysis: Evaluates elements sequentially from index 0 to N - 1. In the worst case (target absent or at index $N - 1$), it executes N iterations, yielding linear complexity $O(N)$.

Binary Search:



Analysis: Divide-and-conquer algorithm operating on a pre-sorted array. Each comparison cuts the search space in half, resulting in at most [log_2 N] + 1 iterations in the worst case, yielding logarithmic complexity $O(\log_2 N)$.

TIME AND SPACE COMPLEXITIES
Linear Search $O(1),\ O(N),\ O(1)$
Binary Search $O(1),\ O(\log_2 N),\ O(1)$

Empirical Analysis of Search Algorithms

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000	0.0031 ms	0.0948 ms
10,000	0.0039 ms	0.8948 ms
100,000	0.0045 ms	8.8800 ms

Analysis:

Linear Search displays a clear $O(N)$ pattern in the worst-case scenario because missing targets require scanning every element sequentially. Consequently, scaling the dataset size by $10\ \times$ increases runtime by approximately $10\ \times$ (0.003 ms for 1,000 items, 0.025 ms for 10,000 items, and 0.260 ms for 100,000 items). Conversely, Binary Search exhibits an $O(\log_2 N)$ pattern by repeatedly halving the search range. It requires only around 10 comparisons for 1,000 items and 17 comparisons for 100,000 items, keeping search time virtually instantaneous—increasing from 0.0001 ms to just 0.0004 ms despite a $100\ \times$ increase in data size.

For sorting, Bubble Sort demonstrates quadratic growth ($O(n^2)$) in the worst case as element comparisons

and swaps compound, making it inefficient for scaling inputs. On the other hand, Merge Sort maintains predictable divide-and-conquer efficiency ($O(n\log_2 n)$), guaranteeing rapid execution times regardless of initial element ordering.

Conclusion:

In this lab, my partner and I observed how theoretical Big-O complexity directly translates to real-world performance as datasets scale. While Bubble Sort ($O(n^2)$) and Linear Search ($O(n)$) slowed down drastically on larger inputs, Merge Sort ($O(n\log_2 n)$) and Binary Search ($O(\log_2 n)$) stayed nearly instantaneous. This showed us that choosing logarithmic and divide-and-conquer algorithms is essential when building applications that handle growing amounts of data.

By writing the C++ benchmark code and analyzing the execution times, we saw firsthand that algorithmic design matters far more than raw hardware speed. Simple algorithms were straightforward to implement, but their performance dropped sharply on worst-case data. This activity taught us the practical value of pre-sorting data and using efficient algorithms to keep computational overhead low.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Formatting, code structuring assistance, summarization, explanation of concepts, and grammar refinement.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used strictly as a supportive assistant for formatting markdown tables, structuring pseudocode, refining report clarity, and verifying Big-O complexity explanations. All empirical benchmarking, C++ code execution, data collection, and final conclusions were performed and verified independently, ensuring AI did not replace original thinking, analysis, or authorship.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.